Project 9: Migration from Maven to Gradle

Objective

The goal of this project is to create a standard Maven Java project, verify that it builds and tests correctly using Maven, and then convert it to a Gradle project using the gradle init --type pom command. The migrated project is verified by inspecting the generated Gradle files and confirming the project structure.

Steps :

Step 1: Create a Maven Project

Open the terminal and run the following Maven archetype command to create a new standard Java project:

```
mvn archetype:generate \
  -DgroupId=com.example \
  -DartifactId=MyMavenToGradle \
  -DarchetypeArtifactId=maven-archetype-quickstart \
  -DinteractiveMode=false
```

This creates a new Maven project named MyMavenToGradle with the standard directory structure including src/main/java and src/test/java. The -DinteractiveMode=false flag skips all interactive prompts.

Step 2: Move into the Project Folder

Navigate into the newly created project directory:

```
cd MyMavenToGradle
```

Run the tree command to verify the Maven project structure was created correctly:

```
tree
```

The Maven project contains pom.xml at the root and src/main/java/com/example/App.java along with src/test/java/com/example/AppTest.java.

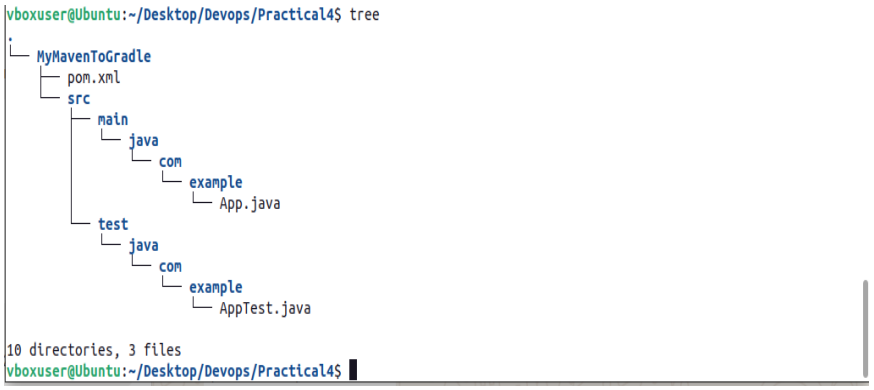


Fig 1: tree output showing the Maven project structure with pom.xml, App.java and AppTest.java

Step 3: Build and Test the Maven Project

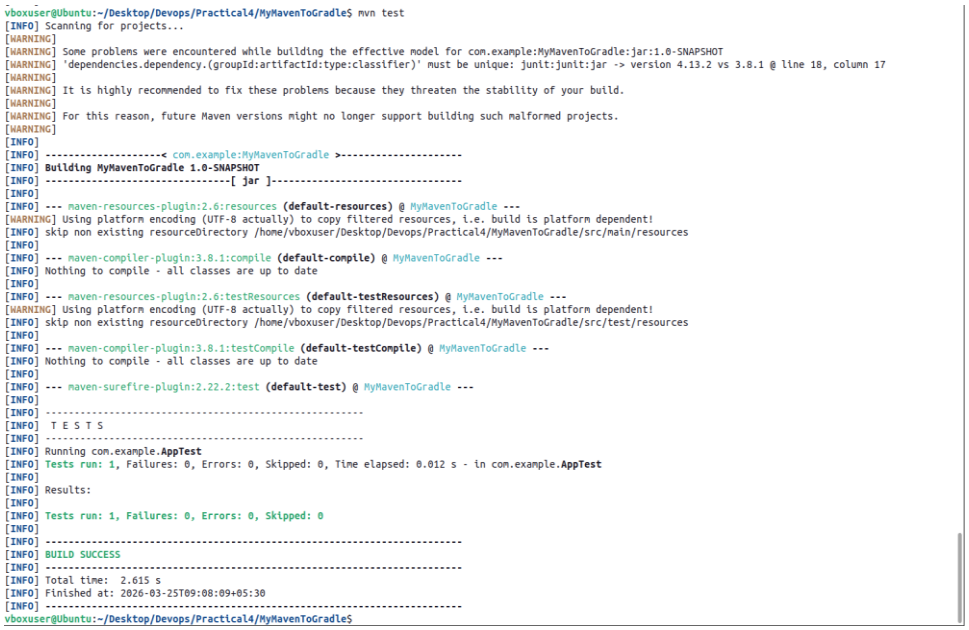
Before migrating, verify that the Maven project builds and all tests pass using:

```
mvn test
```

Maven scans the project, compiles the source and test classes, and runs the JUnit test suite using the Surefire plugin. The output shows:

- Running com.example.AppTest — the test class is located and executed
- Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 — all tests pass
- BUILD SUCCESS — the Maven build completes without errors

This confirms the Maven project is fully functional before the Gradle migration begins.



```
vboxuser@Ubuntu:~/Desktop/Devops/Practical4/MyMavenToGradle$ mvn test
[INFO] Scanning for projects...
[WARNING]
[WARNING] Some problems were encountered while building the effective model for com.example:MyMavenToGradle:jar:1.0-SNAPSHOT
[WARNING] 'dependencies.dependency.(groupId:artifactId:type:classifier)' must be unique: junit:junit:jar -> version 4.13.2 vs 3.8.1 @ line 18, column 17
[WARNING]
[WARNING] It is highly recommended to fix these problems because they threaten the stability of your build.
[WARNING]
[WARNING] For this reason, future Maven versions might no longer support building such malformed projects.
[WARNING]
[INFO]
[INFO] -------------------< com.example:MyMavenToGradle >--------------------
[INFO] Building MyMavenToGradle 1.0-SNAPSHOT
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MyMavenToGradle ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/vboxuser/Desktop/Devops/Practical4/MyMavenToGradle/src/main/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ MyMavenToGradle ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ MyMavenToGradle ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/vboxuser/Desktop/Devops/Practical4/MyMavenToGradle/src/test/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:testCompile (default-testCompile) @ MyMavenToGradle ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- maven-surefire-plugin:2.22.2:test (default-test) @ MyMavenToGradle ---
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.012 s - in com.example.AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  2.615 s
[INFO] Finished at: 2026-03-25T09:08:09+05:30
[INFO] ------------------------------------------------------------------------
vboxuser@Ubuntu:~/Desktop/Devops/Practical4/MyMavenToGradle$
```

Fig 2: mvn test output showing Tests run: 1, Failures: 0 and BUILD SUCCESS for MyMavenToGradle

Step 4: Initialize Gradle from the Maven Project

With the Maven project verified, run the following command from inside the MyMavenToGradle directory to convert it to Gradle:

```
gradle init --type pom
```

Gradle reads the existing pom.xml file and automatically generates the corresponding Gradle build files. The terminal confirms:

- "Maven to Gradle conversion is an incubating feature." — Gradle's built-in Maven importer is active
- "BUILD SUCCESSFUL in 6s" with 2 actionable tasks — conversion completed successfully

Running ls before the conversion shows the original Maven files (gradle, gradlew, gradlew.bat, pom.xml, src, target). After gradle init --type pom, Gradle adds build.gradle and settings.gradle to the project.

```
vboxuser@Ubuntu:~/Desktop/Devops/Practical4/MyMavenToGradle$ ls
gradle  gradlew  gradlew.bat  pom.xml  src  target
vboxuser@Ubuntu:~/Desktop/Devops/Practical4/MyMavenToGradle$ gradle init --type pom

> Task :init
Maven to Gradle conversion is an incubating feature.


BUILD SUCCESSFUL in 6s
2 actionable tasks: 1 executed, 1 up-to-date
vboxuser@Ubuntu:~/Desktop/Devops/Practical4/MyMavenToGradle$
```

Fig 3: ls and gradle init --type pom output — Maven to Gradle conversion BUILD SUCCESSFUL in 6s

Step 5: Verify the Migrated Project Structure

Run tree again to inspect the full project structure after Gradle migration:

```
tree
```

The converted project now contains both Maven and Gradle build files side by side. Key additions after conversion:

- build.gradle — Gradle build file generated from pom.xml
- settings.gradle — Gradle project settings file
- gradle/wrapper/ — contains gradle-wrapper.jar and gradle-wrapper.properties
- gradlew / gradlew.bat — Gradle wrapper scripts for Linux and Windows

The original Maven files are preserved: pom.xml remains and the target/ directory (created during mvn test) is still present with the compiled classes, JAR, and Surefire test reports.

```
vboxuser@Ubuntu:~/Desktop/Devops/Practical4/MyMavenToGradle$ tree
.
├── build.gradle
├── gradle
│   └── wrapper
│       ├── gradle-wrapper.jar
│       └── gradle-wrapper.properties
├── gradlew
├── gradlew.bat
├── pom.xml
├── settings.gradle
├── src
│   ├── main
│   │   └── java
│   │       └── com
│   │           └── example
│   │               └── App.java
│   └── test
│       └── java
│           └── com
│               └── example
│                   └── AppTest.java
└── target
    ├── classes
    │   └── com
    │       └── example
    │           └── App.class
    ├── generated-sources
    │   └── annotations
    ├── generated-test-sources
    │   └── test-annotations
    ├── maven-archiver
    │   └── pom.properties
    ├── maven-status
    │   └── maven-compiler-plugin
    │       ├── compile
    │       │   └── default-compile
    │       │       ├── createdFiles.lst
    │       │       └── inputFiles.lst
    │       └── testCompile
    │           └── default-testCompile
    │               ├── createdFiles.lst
    │               └── inputFiles.lst
    ├── MyMavenToGradle-1.0-SNAPSHOT.jar
    ├── surefire-reports
    │   ├── com.example.AppTest.txt
    │   └── TEST-com.example.AppTest.xml
    └── test-classes
        └── com
            └── example
                └── AppTest.class

30 directories, 19 files
```

Fig 4: tree output after migration showing both Gradle files (build.gradle, gradlew) and original Maven files (pom.xml, target/) coexisting

Step 6: Inspect the Generated build.gradle

Open build.gradle to verify that the Maven dependencies from pom.xml have been correctly translated:

```
cat build.gradle
```

The generated build.gradle typically contains:

- apply plugin: 'java' - Java compilation support (equivalent to Maven's default Java lifecycle)
- repositories - Maven Central declared as the dependency repository
- dependencies - JUnit dependency converted from the pom.xml <dependencies> section
- group, version - project group and version carried over from pom.xml